Final Project Report

Theory and Practice of Deep Learning

Name	ID	Contribution
Jona Patig	1011223	Background Research, Data cleaning, Model Training
Chinnakonda Sreeranjith Devasreshtha	1007868	Data Preprocessing, Model Architecture
Chia Yu Xin, Reese	1007994	Training and Evaluation, Final Model

Github Link: https://github.com/reese232/DeepLearning_Project

Drive Link: https://drive.google.com/drive/folders/1NEqy270wGX-fWen06xXojO2GVVNIOPuH?usp=drive_link

Table of Contents

Table of Contents 2
Introduction 3
Background Research 3
Problem Statement 3
Dataset 5
Chosen Dataset 5
Cleaning Data 6
Normalizing data 7
Deep Learning Model 9
Model Architecture 9
Base model 9
Feature Extractor 9
Model Training 11
During each epoch 11
Training Loop 12
Hyperparameter Tuning 12
First version (Best results) 12
Second version 13
Third version 14
Observations 14
Model Performance 15
Comparison with State-of-the-Art Models 16
Conclusion 17
Summary of Findings 17
Alternative Methods and Modifications Attempts 18
Observed Issues 19
Possible Future Amendments to improve Model 19
Bibliography 20
Appendix 21
Evidence of Model Malfunctioning: 21

Introduction

Background Research

With the current rapid growth of AI, AI-generated fakes, or deepfakes, of all sorts are becoming increasingly realistic and convincing. One of these types of fakes are AI-generated voices, or “spoofs”. Around 2022-2023, the landscape of these spoofs has shifted immensely, with the release of large language models (LLMs) such as GPT-4, Claude, and LLaMA [1]. The swift growth and development of transformer models as a whole (the core architecture of LLMs) led to text to speech (TTS) systems that could synthesize expressive, fluid speech in many different languages, including emotions, and acoustic styles [1]. On top of this, Neural codec language models could now clone specific voices, copying a speaker's timbre and emotion using only a couple of seconds of audio [2]. Already by 2023, two studies have shown that these models have become so sophisticated that a broad sample of the population would more frequently fail to distinguish a spoofed voice from a real one than not [3], [4]. While this can lead to positive effects on the tech industry and user experiences as a whole, it imposes new levels of threats when approached with malicious intent.

Scam calls have been around for over half a century, but over the past 2 decades robocalls (automated calls using computer voices) have become an evermore frequent method of conducting these scams. Phone scams as a whole cause frustration, exploit vulnerable populations (most prominently, the elderly), and destroy the utility and trustworthiness of the global phone network, employing it to illegally profit using deceit [5]. In 2022, an estimated 68.4 million people fell victim to phone scams worldwide, and in the first half of 2023, Americans alone received ~31 billion robocalls, leading to a total loss of $46 billion (USD) [6]. Robocalls have become so common, that it is estimated that they now nearly outnumber legitimate total phone calls [5], [7]. This massive scam industry, combined with the aforementioned improvement of spoofed voices poses great threats to the general public. Integration of TTS with LLMs makes it possible to build fully autonomous, real-time voice agents, the likes of which robocalls have previously not been able to achieve [2]. Unlike traditional robocalls, autonomous AI agents can maintain dialogue in memory, dynamically adapt their responses (context-aware), and employ targeted, researched deceptive persuasion techniques [1], [6]. As the automated scam industry is snowballing into something incredibly dangerous, the search for countermeasures leads directly into this project's problem statement.

Problem Statement

Of course developers of these language models are aware of these dangers, and thus have added pieces into the systems’ architectures that are meant to prevent

malicious use. However, despite countless efforts to safeguard published LLMs from generating potentially dangerous content, research recreations have shown that by deploying strategies such as decomposing prompts, disguising prompts, or incrementally delivering prompts, LLMs can consistently be brought to outputting the harmful content it is programmed against [1]. The problem in its current state is that:

a. People are unable to consistently distinguish spoofed from real voices in fraudulent phone calls, causing billions of dollars in losses
b. Tackling the problem at its source (the models generating deceitful content) is merely an obstacle to the offenders, but workarounds are widespread

Therefore, it's crucial to build technologies that can detect spoofs at a rate that hopefully comes close to that of the improvement of the models generating them. For this project, the aim is to build a deep learning model that can successfully distinguish spoofs from real audio clips at least about 70 percent of the time due to the complexity of the task, assuming spoofs to be an anomaly at an occurrence rate of 5%.

Folder Contents

```
DL Project/
   flac_t/                    # training audio
      T_0000001.flac
      T_0000002.flac
      ... and 54,000 more files
   flac_d/                     # test audio
      D_0000001.flac
      ... and 15,000 more files
   ASVspoof5.train.metadata.txt   # training labels
   ASVspoof5.eval.metadata.txt    # test labels
   outputs_asvspoof/     #data cleaning outputs
         dev_metadata_filtered.txt
         train_metadata_filtered.txt
         dev_missing_entries.txt
         train_missing_entries.txt
         dev_95_5_cleaned.txt
         train_95_5_cleaned.txt
         train_95_5.txt
         test_95_5.txt
         val_95_5.txt
saved_models/ #hyperparameter tuning
saved_models_v2/ #hyperparameter tuning
saved_models_v3/ #hyperparameter tuning
saved_models_v6/ #hyperparameter tuning
saved_models_v7/ #hyperparameter tuning
DL_Model_v1.ipynb #best model
DL_Model_v2.ipynb
DL_Model_v3.ipynb
DL_Model_direct_test_epoch31.ipynb #best parameters and model
```

Package Dependencies and Imports

```
!pip -q install torch torchaudio scikit-learn pandas matplotlib tqdm optuna

import json
import math
import os
import random
from collections import Counter
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.nn.functional as F
import torchaudio
import torchaudio.transforms as T
from torch.utils.data import Dataset, DataLoader, WeightedRandomSampler

from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score,
    f1_score,
    roc_auc_score,
    average_precision_score,
    precision_score,
    recall_score,
    balanced_accuracy_score,
    confusion_matrix,
    roc_curve,
    precision_recall_fscore_support,
)
from tqdm.auto import tqdm
```

Dataset

Chosen Dataset

While many datasets for this exact problem exist, the most credible sources seem to be competition datasets. The largest competition in this field is ASV spoof ("Automatic Speaker Verification and Spoofing Countermeasures Challenge"), whose

fifth and latest competition took place in 2024, thus this being the chosen dataset for this project. It consists of ~324,000 .flac files, split into a train and a dev (both validation and testing) dataset, both containing bonafide and spoofed samples, with “data sourced from the MLS English dataset… from more than 4k speakers, recorded with diverse devices [and]... using the latest TTS and VC algorithms” [8]. More information on the male/female splits and exact TTS and VC algorithms used can be found in the competition paper itself ([8]).

Cleaning Data

The ASVspoof 5 dataset had a lot more spoofed data points than bonafide ones, which was in issue for this specific project as we are assuming the “anomaly” to be spoofs at 5% of the total training data. To adjust for this, the following helper function was implemented:

```
def standardize_raw_split_df(
    df: pd.DataFrame,
    audio_dir: Path,
    file_col: int,
    file_mode: str,
    label_col: int,
) -> pd.DataFrame:
    """
    Convert a raw whitespace-separated metadata DataFrame into a clean
    standardised DataFrame with columns:
        file_id | label_text | label (int) | audio_path
    """
    out = pd.DataFrame()
    out["file_id"]    = df[file_col].astype(str).str.strip()
    out["label_text"] = df[label_col].astype(str).str.lower().str.strip()
    out["label"]      = out["label_text"].map({"bonafide": 0, "spoof": 1})
    out["audio_path"] = out["file_id"].apply(
        lambda fid: str(audio_dir / f"{fid}.flac")
    )
    # Drop rows with unrecognised label values
    n_before = len(out)
    out = out.dropna(subset=["label"]).reset_index(drop=True)
    out["label"] = out["label"].astype(int)
    if len(out) < n_before:
        print(f"[standardize] Dropped {n_before - len(out)} rows with
unrecognised labels.")
    return out
```

The function uses the metadata file of the dataset in order to load the list of files and labels. With these two dataframes, a target percentage of bonafide is set (“bonafide_ratio”), and the maximum target number of spoofed files (max_spoof) is calculated from that. Using pandas’ df.sample method, n random samples are taken from the spoofed dataframe, where n is equal to max_spoof. The sampled version of the bonafide dataframe is simply a copy of the original bonafide dataframe, unless there are too few spoofed samples, in which case the above mentioned methods would be applied to bonafide instead (which will never be the case as long as the original ASVspoof 5 dataset is used). Finally, the sampled dataframes are combined using .concat, and then shuffled in the output by simple “.sample”-ing at a rate of 100% (“frac=1.0”). This was done for both the train and dev dataset. The dev dataset was then split into testing and validation data with a 0.5 split using sklearn's train_test_split.

It must be said that the actual output of this helper function is not a dataset of .flac files, but rather the metadata list of filenames as strings, which are later used in “torchaudio.load(audio_path)” as the audio path. This saves a lot of unnecessary processing of large files when it is not yet required.

The final sizes for the training, validation, and testing dataframes were:

	Training Data	Validation Data	Testing Data
Bonafide	18797	15191	15192
Spoof	989	800	799

Normalizing data

In order to make it possible to predict using a convolutional 2D based model as planned (more on this in Model Architecture), the data is pre-processed. While doing so, it is also normalized in order to assist gradient descent by avoiding extreme ranges of datapoints. This is done in 2-ish steps.

Firstly, “load_and_fix_length”. The files are set to mono in order to end up with a single spectrogram (new dimension[0] (“dim=0” → number of channels) is taken as the average of current ones). Torchaudio's .resample is used to set the waveform to a certain sample rate in order to match the entire dataset. With a consistent sample rate, “max_samples” number of samples are taken from the waveform in order to end up with all data points of a consistent length. If the waveform is too short, it is padded to reach max_samples using torch's .pad().

```python
def load_and_fix_length(
    audio_path: str,
    target_sr: int = SAMPLE_RATE,
    max_samples: int = MAX_SAMPLES,
) -> torch.Tensor:
    waveform, sr = torchaudio.load(audio_path)
    if waveform.shape[0] > 1:                          # stereo → mono
        waveform = waveform.mean(dim=0, keepdim=True)
    if sr != target_sr:
        waveform = torchaudio.functional.resample(waveform, sr, target_sr)
    n = waveform.shape[-1]
    if n < max_samples:
        waveform = torch.nn.functional.pad(waveform, (0, max_samples - n))
    else:
        waveform = waveform[:, :max_samples]
    return waveform
```

Secondly, the waveforms are converted into a MelSpectrogram, by applying a Fast Fourier Transform to convert audio into frequency bins, then mapping those frequencies onto the Mel scale (humans don't hear linearly → 40Hz is not perceived half as low as 80Hz. Mel scale mimics human hearing, or the way that we interpret pitches). Lastly, the amplitude is converted to decibels as its a log scale, completing the log-mel spectrogram, and normalizing amplitudes across the dataset.

```python
class LogMelFeatureExtractor(nn.Module):
    def __init__(self, sample_rate=16_000, n_mels=128, n_fft=1024,
hop_length=160):
        super().__init__()
        self.mel = torchaudio.transforms.MelSpectrogram(
            sample_rate=sample_rate,
            n_fft=n_fft,
            hop_length=hop_length,
            n_mels=n_mels,
            power=2.0,
        )
        self.db = torchaudio.transforms.AmplitudeToDB(stype="power")

    def forward(self, waveform: torch.Tensor) -> torch.Tensor:
        return self.db(self.mel(waveform))
```

In order to make this data usable by the model, the SpoofDataset class is created, returning each sample in the shape of features (log-mel spectrogram), label, row number upon a __getitem__ call.

```
class SpoofDataset(Dataset):
    def __init__(self, df: pd.DataFrame, feature_extractor: nn.Module):
        self.df                 = df.reset_index(drop=True)
        self.feature_extractor = feature_extractor

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row      = self.df.iloc[idx]
        waveform = load_and_fix_length(row["audio_path"])
        features = self.feature_extractor(waveform)          # [1, n_mels,
time]
        label    = torch.tensor(row["label"], dtype=torch.long)
        return features, label, row["file_id"]
```

Deep Learning Model

Model Architecture

Base model

SpoofCNN is a 2D convolutional neural network that classifies each utterance as bonafide or spoof. Each audio clip is first converted into a log-mel spectrogram, giving a single-channel feature map of shape [B, 1, F, T], where F is the number of mel frequency bins and T is the number of time frames. Treating the spectrogram as an image is well-motivated for this task. Spoofing artifacts such as vocoder traces, unnatural harmonic textures, and replay distortions appear as localized patterns in the time-frequency domain, and convolutional layers are well-suited to detect exactly these kinds of local structures.

Feature Extractor

The feature extractor is made up of three stacked ConvBlocks. Each block runs two 3x3 convolutions, followed by batch normalization, ReLU activation, 2x2 max pooling, and dropout. Using two convolutions per block, in the style of VGG networks, gives the model more representational power at each resolution without adding too many parameters. Batch normalization helps stabilize training across the diverse recording conditions in ASVspoof 5. Max pooling halves the time and frequency resolution at each stage, which grows the receptive field and reduces computation.

Channel depth doubles across the three blocks, going from 32 to 64 to 128, so that deeper layers can capture more abstract spoof cues.

For an input spectrogram of shape [B, 1, 80, 400], the dimensions evolve as follows through the feature extractor:

Input: [B, 1, 80, 400,] After ConvBlock 1: [B, 32, 40, 200], After ConvBlock 2: [B, 64, 20, 100], After ConvBlock 3: [B, 128, 10, 50]

Each ConvBlock doubles the channel depth while halving both the frequency and time dimensions through max pooling.

Global Pooling

Because utterance lengths vary across the dataset, we apply adaptive average pooling after the final ConvBlock. This collapses the feature map into a fixed 128-dimensional vector regardless of how long the input is, so no padding or truncation is needed.

After AdaptiveAvgPool2d: [B, 128, 1, 1] After Flatten: [B, 128]

Classification Head

The classification head is a small MLP. It maps the 128-dimensional embedding to 64 dimensions with a ReLU and dropout, then produces two output logits. The head is kept small intentionally to reduce overfitting given the class imbalance in the data.

After Linear(128, 64) + ReLU + Dropout: [B, 64] After Linear(64, 2): [B, 2]

```
class ConvBlock(nn.Module):
    def __init__(self, in_ch: int, out_ch: int, dropout: float = 0.2):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(in_ch,  out_ch, kernel_size=3, padding=1),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, kernel_size=3, padding=1),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2),
            nn.Dropout(dropout),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.block(x)
```

```python
class SpoofCNN(nn.Module):
    def __init__(self, base_channels: int = 32, dropout: float = 0.3,
num_classes: int = 2):
        super().__init__()
        c1, c2, c3 = base_channels, base_channels * 2, base_channels * 4

        self.features = nn.Sequential(
            ConvBlock(1,  c1, dropout=dropout / 2),
            ConvBlock(c1, c2, dropout=dropout / 2),
            ConvBlock(c2, c3, dropout=dropout),
        )
        self.pool = nn.AdaptiveAvgPool2d((1, 1))
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(c3, c2),
            nn.ReLU(inplace=True),
            nn.Dropout(dropout),
            nn.Linear(c2, num_classes),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.classifier(self.pool(self.features(x)))


model = SpoofCNN(base_channels=32, dropout=0.3).to(DEVICE)
print(model)
total_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"\nTrainable parameters: {total_params:,}")
```

Alternative Model Architecture Attempted(V2):

Model Training

During each epoch

The loss function used to train the model is Cross Entropy Loss. The optimizer used is the Adam optimizer with L2 penalty and learning rate decay, where the learning rate is reduced by a factor of 0.5 and has a patience of 3 epochs. The gradient norm of the parameters are also clipped to a maximum of 1.0 to prevent exploding gradients.

Training Loop

In the training loop, for each epoch, the model is trained once on the training dataset and evaluated using the validation dataset. The best threshold for predicting a sample as Spoof is also calculated based on Spoof F1-score. If the Spoof F1-score improved, the model's state is saved along with the values of its hyperparameters and resulting metrics. Early stopping is triggered when there is no improvement in the best Spoof F1-score after 5 epochs.

Hyperparameter Tuning

Hyperparameter tuning was done using Optuna to maximise Spoof F1-score. The hyperparameters are:

- Learning rate
- Weight decay for the optimiser's L2 regularisation
- Dropout

Parameter sampling was done using the TPE (Tree-structured Parzen Estimator) algorithm. Unpromising trials are pruned based on the trial's best intermediate results compared against the median of previous trials.

First version (Best results)

DL_Model_v1.ipynb,
models saved in saved_models

In the first version, early stopping was triggered if the latest two Spoof F1-scores had a difference less than delta=1e-4 and the latest Spoof F1-score was more than 0.98.

The hyperparameter search space is

- Learning rate: any float in logarithmic space in the range of [1e-5, 1e-2]
- Weight decay: any float in logarithmic space in the range of [1e-6, 1e-2]
- Dropout: any float in the range [0.1, 0.5]

The tuning was set for 3 trials with 50 epochs each.

However, we found that the results were stagnating for quite a while before the early stopping triggered. Referring to Figure 1 in the Appendix, the model was overfitting, causing non-optimal parameters for the best model. This could have been due to lack of early stopping and pruning in the initial training, as complex and deep neural networks are more prone to overfitting. Therefore, measures to prevent such cases are required in our model. The global best model for this version had values of:

Epoch 31/50 | train_loss=0.0388 | val_loss=1.0364 | val_acc=0.9784 | val_f1=0.7567 | val_auc=0.8436 | best_thr=0.06 | Spoof recall: 0.6725

```
===== Test Set Results =====

Threshold : 0.07
Loss      : 2.2285
Accuracy  : 0.9684
F1        : 0.6327
ROC-AUC   : 0.8683
EER       : 0.1852
```

Second version

DL_Model_v2.ipynb,
models saved in saved_models

In the second version, early stopping was changed to have a patience of 5 epochs (as described under Training Loop).

The hyperparameter search space is the same as in the first version.

The tuning was set for 3 trials with 35 epochs each.

However, the model was not doing as well as we hoped. The global best model for this version had values of:

Epoch 06/35 | train_loss=0.0151 | val_loss=2.0778 | val_acc=0.9714 | val_f1=0.6696 | val_auc=0.8811 | best_thr=0.07 | Spoof Recall: 0.5444

```
===== Test Set Results =====

Threshold            : 0.07
Loss                 : 2.2285
Accuracy             : 0.9684
Balanced Accuracy    : 0.7676
Spoof Precision      : 0.7552
Spoof Recall         : 0.5444
Spoof F1             : 0.6327
Bonafide Precision   : 0.9764
Bonafide Recall      : 0.9907
Bonafide F1          : 0.9835
Macro F1             : 0.8081
Weighted F1          : 0.9660
ROC-AUC              : 0.8683
PR-AUC (Spoof)       : 0.6681
```

Third version

DL_Model_v3.ipynb,
models saved in saved_models_v3

In the third version, the early stopping implementation remains the same as the second version.

The hyperparameter search space was altered to:

- Learning rate: any number in the set {0.00001, 0.0001, 0.001, 0.01, 0.1}
- Weight decay: any number in the set {0.00001, 0.0001, 0.001, 0.01}
- Dropout: any float in the range [0.2, 0.5] with step=0.1

The tuning was set for 5 trials with 15 epochs each. The global best model for this version had values of:

Epoch 08/15 | train_loss=0.0254 | val_loss=3.4296 | val_acc=0.9744 | val_f1=0.6894 | val_auc=0.8618 | best_thr=0.05 | Spoof Recall: 0.5244

```
===== Test Set Results =====

Threshold            : 0.05
Loss                 : 3.6814
Accuracy             : 0.9713
Balanced Accuracy    : 0.7596
Spoof Precision      : 0.8414
Spoof Recall         : 0.5244
Spoof F1             : 0.6461
Bonafide Precision   : 0.9755
Bonafide Recall      : 0.9948
Bonafide F1          : 0.9850
Macro F1             : 0.8156
Weighted F1          : 0.9681
ROC-AUC              : 0.8484
PR-AUC (Spoof)       : 0.6887
EER                  : 0.2035
```

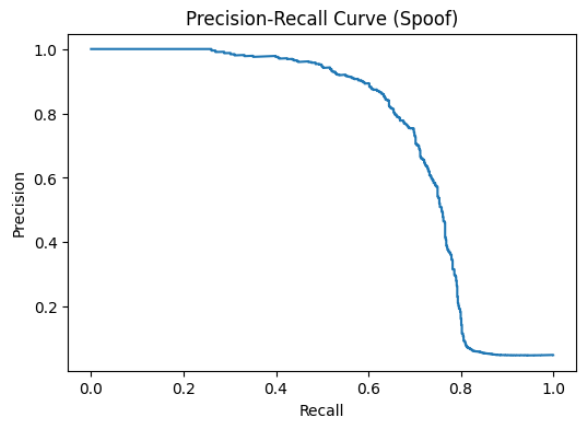
Observations

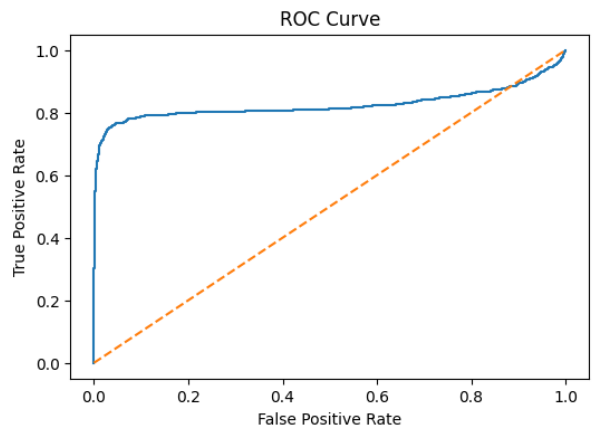
After the three versions of hyperparameter tuning, we found that even though the model was doing relatively well in terms of Spoof F1-score, Spoof Precision (~0.7-0.8) was much higher than Spoof Recall (~0.5-0.6). However, in our case, we should prioritise Recall over Precision since false negatives are more harmful than false positives. This observation prompted us to explore ways to improve our Recall, as described in Alternative Methods and Modifications.

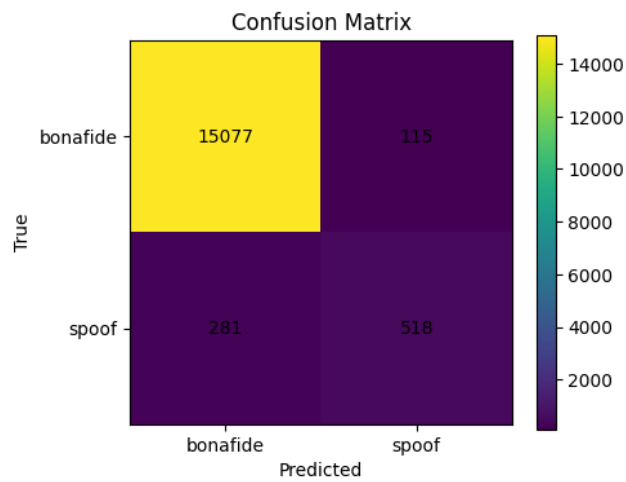
Model Performance

The best model performance we received was from DL_Model_v1. To test and run the code with the best parameters, run DL_Model_direct_test_epoch31.ipynb, output of the code would be present as well.
To assess which model performed best, we calculated and compared the spoof recall and precision, and in the end the min dcf and EER. Amongst all measures, this model and output was the best.

Metric	Result
Threshold	0.06
Loss	1.0690
Accuracy	0.9752
Balanced Accuracy	0.8204
Spoof Precision	0.8183
Spoof Recall	0.6483
Spoof F1	0.7235
Bonafide Precision	0.9817
Bonafide Recall	0.9924
Bonafide F1	0.9870
Macro F1	0.8553
Weighted F1	0.9739
ROC-AUC	0.8253
PR-AUC (Spoof)	0.7293
min df	0.6883
EER	0.2059







Comparison with State-of-the-Art Models

As the dataset stems from a competition, it's possible to compare the results both to the baseline model(s), as well as the competition finalists. It is noted that the baseline models (there are two listed in the paper, both with the same results) achieve a minDCF score of ~>0.7 and EER of ~29% [8]. With that being said, it is important to note that the authors did not publish other metrics for their model, as in this specific case, these metrics are the most meaningful and telling about this specific type of dataset and problem. This is because:

- EER (Equal Error Rate) is the error rate at the exact threshold where false positives would equal false negatives. In the competition this metric is very straight forward, as there are a lot of spoofed and bonafide samples in the dataset. While this isn't the most telling metric for this project specifically, it gives an error value in a tougher environment than it was built in.

- minDCF (Minimum Detection Cost Function) is a cost-weighted error metric that accounts for how rare spoofs are within the dataset, and how much each error costs. In our code, the cost for missing a spoof is 1, and the cost for wrongly labeling a bonafide sample is 10. While this goes against the dataset, these are the values used within the competition, lining up with the target context of the competition models. The models will be used to shut down calls before reaching a human, and shutting down too many calls wrongly would break down the phone network. However, this weighted metric will give the most accurate indicator of the viability of our model. Below, the code for obtaining a minDCF value can be seen with c_miss and c_fa being set in the function definition.

The best model managed to achieve a minDCF score of 0.6883, and an EER of of 20.59%. Both values are improvements on the baseline model of the competition, but not enough to achieve top 5. In order to qualify for this, a minDCF score of <0.5, and an EER of <15% is necessary [8]. However, as mentioned previously, these top competition values were obtained at a much larger number of spoof samples, making these values as a comparison an uphill battle for the project's model. More on the implications of these results below.

Conclusion

Summary of Findings

The best model achieved in this project is that of *DL_Model_v1.ipynb.* The model was built with an F1 optimizer, achieving a final F1 score of 0.98. In the scope of all other training and model attempts, with a spoof recall of ~64.8%, these values are decently high (see Model Architecture and Hyperparameter tuning sections). However, pitting it up against competition models showed some shortcomings (see Comparison with existing models) . With this model beating the competition baseline, it proves that it is effective for the task at hand, but it does not compete with state of the art models. This is most likely due to two reasons.

Firstly, the metrics which were provided for the state of the art models had a slightly different goal in mind than what our model was built on. Setting a high cost for falsely mislabeling bonafide samples (c_fa), and a low cost for missing spoof samples (c_m) sets focus on how impeding the model would be on phone traffic if calls were shut down upon a false positive. Yet our models aimed to flag as many spoofs as possible without flagging everything as a spoof.

Secondly, the model was built on a much more difficult dataset label ratio. While this came up several times already within this report, the decision was made as spoofs are taken as an anomaly with a rate of 5%. This arbitrary split is meant to represent the

average person's phone calls rather than that of a company. The competition dataset on the other hand was built with corporations in mind, that would receive target bot calls very frequently.

The achieved values of the best model still manage to stack up against the competition baseline, proving the overall building and training as a success by displaying its acceptability in a tougher competition environment. This being said, building an anomaly detection model on data that can quite literally be generated infinitely (spoofed audio) but sticking to the anomaly occurrence rate is simply a massive disadvantage that would most likely not occur at industry standards.

Alternative Methods and Modifications Attempts

DL_Model_v2_improved.ipynb

Five modifications were introduced to the training pipeline to improve spoof recall beyond the baseline results.

1. The first change altered the optimization target from generic binary F1 to spoof recall. The original threshold search favoured bonafide accuracy at the 95/5 ratio, so the new search instead maximised spoof recall subject to a minimum precision floor of 0.40, with model checkpointing based on spoof recall directly.
2. The second change replaced CrossEntropyLoss with Focal Loss, which down-weights easy well-classified examples and concentrates gradient updates on hard or rare samples. Alpha and gamma were included in the Optuna search space to find the appropriate imbalance trade-off automatically.
3. The third change introduced a WeightedRandomSampler to produce balanced mini-batches during training. The original setup drew approximately 95% bonafide samples per batch, meaning each gradient update was dominated by the majority class even with loss weighting applied.
4. The fourth change replaced AdaptiveAvgPool2d with Attentive Statistics Pooling, which learns a per-channel attention weight over time frames rather than averaging all frames equally. This preserves temporal structure that may carry spoofing artefacts, and produces a concatenation of weighted mean and standard deviation, doubling the classifier input size from 128 to 256.
5. The fifth change added SpecAugment during training only, applying stochastic frequency and time masking to improve generalisation to unseen spoofing methods.

Observed Issues

The primary failure was a model collapse in which spoof recall approached 1.0 while spoof precision fell to approximately 0.05, meaning the model classified nearly all samples as spoof regardless of content. This occurred when the WeightedRandomSampler and Focal Loss were combined with a very low learning rate, creating a degenerate solution that the optimiser could not escape. The sampler was subsequently made less aggressive, targeting a 20% spoof ratio per batch rather than 50%, allowing Focal Loss to handle the primary imbalance correction while the sampler ensured consistent spoof exposure. Although initially the recall was high, precision was significantly lower, leading to us not continuing with the model architecture.

```
[I 2026-04-17 18:01:10,424] A new study created in memory with name: no-name-79871e29-2531-4982-b9f6-9aefbd91407c
Epoch 01/10 | train_loss=0.0253 | val_loss=0.0329 | acc=0.9315 | spoof_rec=0.8013 | spoof_prec=0.4065 | macro_f1=0.7512 | roc_auc=0.8949 | thr=0.30
  *** Improved (spoof_recall=0.8013), saved -> /content/drive/MyDrive/DL Project/saved_models_v2/lr0.000315_decay0.007114/epoch01_spoof_rec0.8013.pt
Epoch 02/10 | train_loss=0.0118 | val_loss=0.0327 | acc=0.9345 | spoof_rec=0.7588 | spoof_prec=0.4155 | macro_f1=0.7509 | roc_auc=0.8822 | thr=0.31
Epoch 03/10 | train_loss=0.0107 | val_loss=0.0708 | acc=0.9414 | spoof_rec=0.3475 | spoof_prec=0.4012 | macro_f1=0.6708 | roc_auc=0.7285 | thr=0.70
[I 2026-04-17 22:22:48,829] Trial 0 finished with value: 0.7511741505761 92 and parameters: {'lr': 0.0003148911647956 8613, 'weight_decay': 0.007114476009343422 5, 'dropout': 0.4, 'focal_alpha': 0.8, 'focal_gamma': 1.0}. Best is
Epoch 04/10 | train_loss=0.0103 | val_loss=0.0552 | acc=0.9379 | spoof_rec=0.5550 | spoof_prec=0.4107 | macro_f1=0.7196 | roc_auc=0.7844 | thr=0.67
Early stopping: no spoof_recall improvement for 3 epochs.
  *** New global best: macro_f1=0.7512 | spoof_recall=0.8013 | params={'lr': 0.0003148911647956 8613, 'weight_decay': 0.007114476009343422 5, 'dropout': 0.4, 'focal_alpha': 0.8, 'focal_gamma': 1.0}
Epoch 01/10 | train_loss=0.0158 | val_loss=0.3169 | acc=0.0624 | spoof_rec=0.9912 | spoof_prec=0.0503 | macro_f1=0.0611 | roc_auc=0.7102 | thr=0.70
  *** Improved (spoof_recall=0.9912), saved -> /content/drive/MyDrive/DL Project/saved_models_v2/lr0.000042_decay0.000015/epoch01_spoof_rec0.9912.pt
Epoch 02/10 | train_loss=0.0056 | val_loss=0.5694 | acc=0.0521 | spoof_rec=0.9975 | spoof_prec=0.0500 | macro_f1=0.0499 | roc_auc=0.6678 | thr=0.70
  *** Improved (spoof_recall=0.9975), saved -> /content/drive/MyDrive/DL Project/saved_models_v2/lr0.000042_decay0.000015/epoch02_spoof_rec0.9975.pt
Epoch 03/10 | train_loss=0.0040 | val_loss=0.6702 | acc=0.0522 | spoof_rec=0.9988 | spoof_prec=0.0501 | macro_f1=0.0501 | roc_auc=0.6718 | thr=0.70
  *** Improved (spoof_recall=0.9988), saved -> /content/drive/MyDrive/DL Project/saved_models_v2/lr0.000042_decay0.000015/epoch03_spoof_rec0.9988.pt
Epoch 04/10 | train_loss=0.0033 | val_loss=0.6623 | acc=0.0572 | spoof_rec=0.9962 | spoof_prec=0.0502 | macro_f1=0.0555 | roc_auc=0.6719 | thr=0.70
Epoch 05/10 | train_loss=0.0024 | val_loss=0.7091 | acc=0.0522 | spoof_rec=0.9988 | spoof_prec=0.0501 | macro_f1=0.0501 | roc_auc=0.7202 | thr=0.69
Epoch 06/10 | train_loss=0.0021 | val_loss=0.8060 | acc=0.0530 | spoof_rec=1.0000 | spoof_prec=0.0502 | macro_f1=0.0509 | roc_auc=0.6797 | thr=0.70
  *** Improved (spoof_recall=1.0000), saved -> /content/drive/MyDrive/DL Project/saved_models_v2/lr0.000042_decay0.000015/epoch06_spoof_rec1.0000.pt
[W 2026-04-17 23:51:17,660] Trial 1 failed with parameters: {'lr': 4.207053950287933e-05, 'weight_decay': 1.493656855461762e-05, 'dropout': 0.5, 'focal_alpha': 0.8, 'focal_gamma': 2.5} because of the following error: Keyboard
Traceback (most recent call last):
```

Possible Future Amendments to improve Model

Several directions could meaningfully extend this work beyond the current implementation. Due to computational constraints with the T4 GPU on Google Colab and session timeouts, more complex architectures could not be explored within the scope of this project, and the Optuna search was restricted to a small number of trials, limiting confidence in the reported optimal hyperparameters.

End-to-end models operating directly on raw waveforms, such as RawNet2 or Wav2Vec2-based approaches, have shown strong results on ASVspoof benchmarks by learning their own low-level feature representations, capturing phase and temporal information that mel spectrograms discard. Self-supervised pretraining on large unlabelled speech corpora is another promising direction, as fine-tuned models have achieved state-of-the-art EER on ASVspoof 5 without requiring large labelled datasets.

A more principled treatment of class imbalance would involve training on the natural dataset distribution rather than the artificial 95/5 ratio. For robustness, adversarial augmentation or domain generalisation techniques would better prepare the model for unseen spoofing methods, which is the more realistic deployment scenario. Finally, ensemble methods combining multiple independently trained models or additional modalities such as prosody and speaker embeddings could improve reliability beyond what a single model achieves.

Bibliography

[1] S. Badhe, “ScamAgents: How AI Agents Can Simulate Human-Level Scam Calls,” Dec. 09, 2025, *arXiv*: arXiv:2508.06457. doi: 10.48550/arXiv.2508.06457.

[2] K. Kaewtawee, W. Modecrua, K. Pachtrachai, and T. Kraisingkorn, “Cloning a Conversational Voice AI Agent from Call\,Recording Datasets for Telesales,” 2025, *arXiv*. doi: 10.48550/ARXIV.2509.04871.

[3] C. Sun, S. Jia, S. Hou, and S. Lyu, “AI-Synthesized Voice Detection Using Neural Vocoder Artifacts,” in *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, Vancouver, BC, Canada: IEEE, Jun. 2023, pp. 904–912. doi: 10.1109/CVPRW59228.2023.00097.

[4] Z. Khanjani, G. Watson, and V. P. Janeja, “Audio deepfakes: A survey,” *Front. Big Data*, vol. 5, p. 1001063, Jan. 2023, doi: 10.3389/fdata.2022.1001063.

[5] USENIX Association, Ed., *Artifact appendices to the proceedings of the 32nd USENIX Security Symposium: August 9-11, 2023, Anaheim, CA, USA*. Berkeley, CA: USENIX Association, 2023.

[6] M. Pietri, M. Mamei, and M. Colajanni, “Evaluating Technical Countermeasures for Telecom Spam and Scams in the AI Era,” in *2024 22nd International Symposium on Network Computing and Applications (NCA)*, Bertinoro, Italy: IEEE, Oct. 2024, pp. 270–277. doi: 10.1109/NCA61908.2024.00047.

[7] E. Brown, “Over half of the calls people receive now are spam,” *ZD Net*, Apr. 2020, [Online]. Available: https://www.zdnet.com/article/over-half-of-the-calls-people-receive-now-are-spam/

[8] X. Wang *et al.*, “ASVspoof 5: crowdsourced speech data, deepfakes, and adversarial attacks at scale,” in *The Automatic Speaker Verification Spoofing Countermeasures Workshop (ASVspoof 2024)*, ISCA, Aug. 2024, pp. 1–8. doi: 10.21437/ASVspoof.2024-1.

Appendix

Evidence of Model Malfunctioning:

1. Overfitting during Hyperparameter Tuning, even after the validation loss is much worse than the train loss:

```
study = run_optuna_search(n_trials= 30, n_jobs=1)
```

```
[I 2026-04-12 13:09:31,978] A new study created in memory with name: no-name-31ab6f8f-9295-4e3b-91da-e3d77d96d34f
Epoch 01/50 | train_loss=0.1851 | val_loss=1.1143 | val_acc=0.8367 | val_f1=0.3244 | val_auc=0.8442 | best_thr=0.95
Model improved, saving best model to /content/drive/MyDrive/DL Project/saved_models/lr0.0001329291894316216_decay0.006351221010640704/model_lr0.0001329291894316216_decay0.006351221010640704_epoch01.pt...
Epoch 02/50 | train_loss=0.0884 | val_loss=0.7903 | val_acc=0.9774 | val_f1=0.7412 | val_auc=0.8568 | best_thr=0.38
Model improved, saving best model to /content/drive/MyDrive/DL Project/saved_models/lr0.0001329291894316216_decay0.006351221010640704/model_lr0.0001329291894316216_decay0.006351221010640704_epoch02.pt...
Epoch 03/50 | train_loss=0.0614 | val_loss=0.8137 | val_acc=0.9651 | val_f1=0.5842 | val_auc=0.8000 | best_thr=0.71
Epoch 04/50 | train_loss=0.0557 | val_loss=2.1108 | val_acc=0.9666 | val_f1=0.5215 | val_auc=0.8156 | best_thr=0.05
Epoch 05/50 | train_loss=0.0520 | val_loss=2.0170 | val_acc=0.9667 | val_f1=0.5159 | val_auc=0.8354 | best_thr=0.05
Epoch 06/50 | train_loss=0.0498 | val_loss=1.2795 | val_acc=0.9676 | val_f1=0.6433 | val_auc=0.8260 | best_thr=0.09
Epoch 07/50 | train_loss=0.0297 | val_loss=0.7539 | val_acc=0.9681 | val_f1=0.6458 | val_auc=0.8308 | best_thr=0.69
Epoch 08/50 | train_loss=0.0307 | val_loss=1.2466 | val_acc=0.9702 | val_f1=0.6422 | val_auc=0.7924 | best_thr=0.13
Epoch 09/50 | train_loss=0.0288 | val_loss=0.8335 | val_acc=0.9677 | val_f1=0.6189 | val_auc=0.8203 | best_thr=0.73
Epoch 10/50 | train_loss=0.0293 | val_loss=1.2943 | val_acc=0.9667 | val_f1=0.6084 | val_auc=0.8038 | best_thr=0.14
Epoch 11/50 | train_loss=0.0268 | val_loss=1.2639 | val_acc=0.9734 | val_f1=0.6966 | val_auc=0.8267 | best_thr=0.05
Epoch 12/50 | train_loss=0.0233 | val_loss=1.0034 | val_acc=0.9745 | val_f1=0.7217 | val_auc=0.8538 | best_thr=0.12
Epoch 13/50 | train_loss=0.0267 | val_loss=1.6213 | val_acc=0.9724 | val_f1=0.6470 | val_auc=0.8298 | best_thr=0.05
Epoch 14/50 | train_loss=0.0246 | val_loss=0.7383 | val_acc=0.9637 | val_f1=0.5809 | val_auc=0.7816 | best_thr=0.95
Epoch 15/50 | train_loss=0.0230 | val_loss=1.2081 | val_acc=0.9714 | val_f1=0.6525 | val_auc=0.8177 | best_thr=0.13
Epoch 16/50 | train_loss=0.0277 | val_loss=1.0767 | val_acc=0.9717 | val_f1=0.6785 | val_auc=0.8263 | best_thr=0.11
Epoch 17/50 | train_loss=0.0266 | val_loss=1.3229 | val_acc=0.9719 | val_f1=0.6627 | val_auc=0.8153 | best_thr=0.06
Epoch 18/50 | train_loss=0.0289 | val_loss=1.4637 | val_acc=0.9745 | val_f1=0.6767 | val_auc=0.8295 | best_thr=0.05
Epoch 19/50 | train_loss=0.0247 | val_loss=1.0904 | val_acc=0.9717 | val_f1=0.6878 | val_auc=0.8294 | best_thr=0.07
Epoch 20/50 | train_loss=0.0293 | val_loss=1.2406 | val_acc=0.9752 | val_f1=0.7067 | val_auc=0.8396 | best_thr=0.05
Epoch 21/50 | train_loss=0.0294 | val_loss=0.7870 | val_acc=0.9776 | val_f1=0.7482 | val_auc=0.8503 | best_thr=0.19
Model improved, saving best model to /content/drive/MyDrive/DL Project/saved_models/lr0.0001329291894316216_decay0.006351221010640704/model_lr0.0001329291894316216_decay0.006351221010640704_epoch21.pt...
Epoch 22/50 | train_loss=0.0309 | val_loss=1.0420 | val_acc=0.9765 | val_f1=0.7339 | val_auc=0.8373 | best_thr=0.08
Epoch 23/50 | train_loss=0.0328 | val_loss=0.8894 | val_acc=0.9750 | val_f1=0.7208 | val_auc=0.8388 | best_thr=0.15
Epoch 24/50 | train_loss=0.0345 | val_loss=1.4062 | val_acc=0.9746 | val_f1=0.6752 | val_auc=0.8308 | best_thr=0.05
Epoch 25/50 | train_loss=0.0366 | val_loss=1.1252 | val_acc=0.9737 | val_f1=0.6943 | val_auc=0.8309 | best_thr=0.08
Epoch 26/50 | train_loss=0.0332 | val_loss=0.9584 | val_acc=0.9764 | val_f1=0.7328 | val_auc=0.8338 | best_thr=0.10
Epoch 27/50 | train_loss=0.0327 | val_loss=0.9520 | val_acc=0.9759 | val_f1=0.7328 | val_auc=0.8378 | best_thr=0.09
Epoch 28/50 | train_loss=0.0337 | val_loss=1.3011 | val_acc=0.9764 | val_f1=0.7075 | val_auc=0.8399 | best_thr=0.05
Epoch 29/50 | train_loss=0.0380 | val_loss=1.1861 | val_acc=0.9768 | val_f1=0.7298 | val_auc=0.8450 | best_thr=0.05
Epoch 30/50 | train_loss=0.0361 | val_loss=0.9216 | val_acc=0.9769 | val_f1=0.7462 | val_auc=0.8476 | best_thr=0.09
Epoch 31/50 | train_loss=0.0388 | val_loss=1.0364 | val_acc=0.9784 | val_f1=0.7567 | val_auc=0.8436 | best_thr=0.06
Model improved, saving best model to /content/drive/MyDrive/DL Project/saved_models/lr0.0001329291894316216_decay0.006351221010640704/model_lr0.0001329291894316216_decay0.006351221010640704_epoch31.pt...
Epoch 32/50 | train_loss=0.0370 | val_loss=0.9490 | val_acc=0.9735 | val_f1=0.7112 | val_auc=0.8353 | best_thr=0.09
Epoch 33/50 | train_loss=0.0414 | val_loss=0.9847 | val_acc=0.9774 | val_f1=0.7486 | val_auc=0.8426 | best_thr=0.07
```

```
Epoch 34/50 | train_loss=0.0391 | val_loss=0.8627 | val_acc=0.9769 | val_f1=0.7409 | val_auc=0.8410 | best_thr=0.13
Epoch 35/50 | train_loss=0.0422 | val_loss=0.9530 | val_acc=0.9769 | val_f1=0.7398 | val_auc=0.8389 | best_thr=0.09
Epoch 36/50 | train_loss=0.0409 | val_loss=1.0892 | val_acc=0.9755 | val_f1=0.7264 | val_auc=0.8353 | best_thr=0.05
Epoch 37/50 | train_loss=0.0376 | val_loss=0.9836 | val_acc=0.9780 | val_f1=0.7521 | val_auc=0.8425 | best_thr=0.07
Epoch 38/50 | train_loss=0.0445 | val_loss=0.9408 | val_acc=0.9782 | val_f1=0.7523 | val_auc=0.8432 | best_thr=0.09
Epoch 39/50 | train_loss=0.0402 | val_loss=1.1244 | val_acc=0.9779 | val_f1=0.7455 | val_auc=0.8478 | best_thr=0.05
Epoch 40/50 | train_loss=0.0433 | val_loss=1.0239 | val_acc=0.9775 | val_f1=0.7486 | val_auc=0.8426 | best_thr=0.06
Epoch 41/50 | train_loss=0.0403 | val_loss=1.0007 | val_acc=0.9769 | val_f1=0.7402 | val_auc=0.8381 | best_thr=0.07
Epoch 42/50 | train_loss=0.0394 | val_loss=1.0005 | val_acc=0.9772 | val_f1=0.7490 | val_auc=0.8420 | best_thr=0.06
Epoch 43/50 | train_loss=0.0437 | val_loss=1.0814 | val_acc=0.9777 | val_f1=0.7479 | val_auc=0.8406 | best_thr=0.05
Epoch 44/50 | train_loss=0.0389 | val_loss=1.0012 | val_acc=0.9767 | val_f1=0.7377 | val_auc=0.8382 | best_thr=0.07
Epoch 45/50 | train_loss=0.0433 | val_loss=1.0708 | val_acc=0.9775 | val_f1=0.7488 | val_auc=0.8414 | best_thr=0.05
Epoch 46/50 | train_loss=0.0415 | val_loss=1.0383 | val_acc=0.9775 | val_f1=0.7463 | val_auc=0.8408 | best_thr=0.06
Epoch 47/50 | train_loss=0.0458 | val_loss=1.0140 | val_acc=0.9764 | val_f1=0.7354 | val_auc=0.8393 | best_thr=0.07
Epoch 48/50 | train_loss=0.0388 | val_loss=1.0040 | val_acc=0.9761 | val_f1=0.7387 | val_auc=0.8395 | best_thr=0.06
Epoch 49/50 | train_loss=0.0403 | val_loss=1.0284 | val_acc=0.9779 | val_f1=0.7466 | val_auc=0.8412 | best_thr=0.07
[I 2026-04-12 21:36:10,686] Trial 0 finished with value: 0.7566807313642757 and parameters: {'lr': 0.0001329291894316216, 'weight_decay': 0.006351221010640704, 'dropout': 0.39279757672456206}. Best is trial 0 with value: 0.756
Epoch 50/50 | train_loss=0.0437 | val_loss=1.0169 | val_acc=0.9769 | val_f1=0.7439 | val_auc=0.8389 | best_thr=0.06
  *** New global best: F1=0.7567 | params={'lr': 0.0001329291894316216, 'weight_decay': 0.006351221010640704, 'dropout': 0.39279757672456206}
Epoch 01/50 | train_loss=0.1246 | val_loss=3.1299 | val_acc=0.9592 | val_f1=0.4647 | val_auc=0.7773 | best_thr=0.05
Model improved, saving best model to /content/drive/MyDrive/DL Project/saved_models/lr0.0006251373574521745_decay4.2079886696066345e-06/model_lr0.0006251373574521745_decay4.2079886696066345e-06_epoch01.pt...
Epoch 02/50 | train_loss=0.0411 | val_loss=3.8584 | val_acc=0.9664 | val_f1=0.5096 | val_auc=0.7938 | best_thr=0.05
Model improved, saving best model to /content/drive/MyDrive/DL Project/saved_models/lr0.0006251373574521745_decay4.2079886696066345e-06/model_lr0.0006251373574521745_decay4.2079886696066345e-06_epoch02.pt...
Epoch 03/50 | train_loss=0.0291 | val_loss=5.4627 | val_acc=0.9529 | val_f1=0.1110 | val_auc=0.8536 | best_thr=0.05
Epoch 04/50 | train_loss=0.0184 | val_loss=7.5112 | val_acc=0.9522 | val_f1=0.1509 | val_auc=0.7248 | best_thr=0.05
Epoch 05/50 | train_loss=0.0104 | val_loss=3.5701 | val_acc=0.9682 | val_f1=0.5690 | val_auc=0.8607 | best_thr=0.05
Model improved, saving best model to /content/drive/MyDrive/DL Project/saved_models/lr0.0006251373574521745_decay4.2079886696066345e-06/model_lr0.0006251373574521745_decay4.2079886696066345e-06_epoch05.pt...
Epoch 06/50 | train_loss=0.0191 | val_loss=2.3640 | val_acc=0.9138 | val_f1=0.4171 | val_auc=0.7956 | best_thr=0.95
Epoch 07/50 | train_loss=0.0168 | val_loss=1.9084 | val_acc=0.9800 | val_f1=0.7784 | val_auc=0.8652 | best_thr=0.95
Model improved, saving best model to /content/drive/MyDrive/DL Project/saved_models/lr0.0006251373574521745_decay4.2079886696066345e-06/model_lr0.0006251373574521745_decay4.2079886696066345e-06_epoch07.pt...
Epoch 08/50 | train_loss=0.0084 | val_loss=4.1592 | val_acc=0.9664 | val_f1=0.5277 | val_auc=0.8570 | best_thr=0.05
Epoch 09/50 | train_loss=0.0105 | val_loss=2.3811 | val_acc=0.9724 | val_f1=0.7045 | val_auc=0.8619 | best_thr=0.95
Epoch 10/50 | train_loss=0.0193 | val_loss=5.0233 | val_acc=0.9647 | val_f1=0.4896 | val_auc=0.8120 | best_thr=0.05
Epoch 11/50 | train_loss=0.0162 | val_loss=11.4827 | val_acc=0.9530 | val_f1=0.1358 | val_auc=0.7010 | best_thr=0.06
Epoch 12/50 | train_loss=0.0066 | val_loss=4.1921 | val_acc=0.9705 | val_f1=0.6091 | val_auc=0.8572 | best_thr=0.05
Epoch 13/50 | train_loss=0.0036 | val_loss=5.3346 | val_acc=0.9673 | val_f1=0.5456 | val_auc=0.8124 | best_thr=0.05
Epoch 14/50 | train_loss=0.0061 | val_loss=5.0272 | val_acc=0.9672 | val_f1=0.5475 | val_auc=0.8399 | best_thr=0.05
Epoch 15/50 | train_loss=0.0017 | val_loss=6.5104 | val_acc=0.9639 | val_f1=0.4421 | val_auc=0.8387 | best_thr=0.05
Epoch 16/50 | train_loss=0.0005 | val_loss=7.4655 | val_acc=0.9640 | val_f1=0.4611 | val_auc=0.8022 | best_thr=0.05
```

```
Epoch 06/50 | train_loss=0.0191 | val_loss=2.3640 | val_acc=0.9138 | val_f1=0.4171 | val_auc=0.7956 | best_thr=0.95
Epoch 07/50 | train_loss=0.0168 | val_loss=1.9084 | val_acc=0.9800 | val_f1=0.7784 | val_auc=0.8652 | best_thr=0.95
Model improved, saving best model to /content/drive/MyDrive/DL Project/saved_models/lr0.0006251373574521745_decay4.2079886696066345e-06/model_lr0.0006251373574521745_decay4.2079886696066345e-06_epoch07.pt...
Epoch 08/50 | train_loss=0.0084 | val_loss=4.1592 | val_acc=0.9664 | val_f1=0.5277 | val_auc=0.8570 | best_thr=0.05
Epoch 09/50 | train_loss=0.0105 | val_loss=2.3811 | val_acc=0.9724 | val_f1=0.7045 | val_auc=0.8619 | best_thr=0.95
Epoch 10/50 | train_loss=0.0193 | val_loss=5.0233 | val_acc=0.9647 | val_f1=0.4896 | val_auc=0.8120 | best_thr=0.05
Epoch 11/50 | train_loss=0.0162 | val_loss=11.4827 | val_acc=0.9530 | val_f1=0.1358 | val_auc=0.7010 | best_thr=0.06
Epoch 12/50 | train_loss=0.0066 | val_loss=4.1921 | val_acc=0.9785 | val_f1=0.6091 | val_auc=0.8572 | best_thr=0.05
Epoch 13/50 | train_loss=0.0036 | val_loss=5.3346 | val_acc=0.9673 | val_f1=0.5456 | val_auc=0.8124 | best_thr=0.05
Epoch 14/50 | train_loss=0.0061 | val_loss=5.0272 | val_acc=0.9672 | val_f1=0.5475 | val_auc=0.8399 | best_thr=0.05
Epoch 15/50 | train_loss=0.0017 | val_loss=6.5104 | val_acc=0.9639 | val_f1=0.4421 | val_auc=0.8387 | best_thr=0.05
Epoch 16/50 | train_loss=0.0005 | val_loss=7.4655 | val_acc=0.9640 | val_f1=0.4611 | val_auc=0.8022 | best_thr=0.05
Epoch 17/50 | train_loss=0.0021 | val_loss=7.6659 | val_acc=0.9600 | val_f1=0.3629 | val_auc=0.8186 | best_thr=0.05
Epoch 18/50 | train_loss=0.0003 | val_loss=9.4302 | val_acc=0.9595 | val_f1=0.3306 | val_auc=0.7778 | best_thr=0.05
Epoch 19/50 | train_loss=0.0018 | val_loss=11.1996 | val_acc=0.9538 | val_f1=0.1535 | val_auc=0.7480 | best_thr=0.05
Epoch 20/50 | train_loss=0.0013 | val_loss=9.7817 | val_acc=0.9564 | val_f1=0.2590 | val_auc=0.7535 | best_thr=0.05
Epoch 21/50 | train_loss=0.0005 | val_loss=9.1063 | val_acc=0.9573 | val_f1=0.2922 | val_auc=0.7647 | best_thr=0.05
Epoch 22/50 | train_loss=0.0003 | val_loss=8.0207 | val_acc=0.9608 | val_f1=0.3798 | val_auc=0.8090 | best_thr=0.05
Epoch 23/50 | train_loss=0.0002 | val_loss=10.4360 | val_acc=0.9559 | val_f1=0.2276 | val_auc=0.7717 | best_thr=0.05
Epoch 24/50 | train_loss=0.0023 | val_loss=9.1820 | val_acc=0.9583 | val_f1=0.3016 | val_auc=0.7864 | best_thr=0.05
Epoch 25/50 | train_loss=0.0007 | val_loss=10.0699 | val_acc=0.9557 | val_f1=0.2151 | val_auc=0.7918 | best_thr=0.05
Epoch 26/50 | train_loss=0.0000 | val_loss=9.7989 | val_acc=0.9567 | val_f1=0.2527 | val_auc=0.7859 | best_thr=0.05
Epoch 27/50 | train_loss=0.0002 | val_loss=11.3540 | val_acc=0.9539 | val_f1=0.1538 | val_auc=0.7714 | best_thr=0.05
Epoch 28/50 | train_loss=0.0004 | val_loss=9.5870 | val_acc=0.9566 | val_f1=0.2424 | val_auc=0.7985 | best_thr=0.05
Epoch 29/50 | train_loss=0.0000 | val_loss=10.0977 | val_acc=0.9553 | val_f1=0.1993 | val_auc=0.7922 | best_thr=0.05
Epoch 30/50 | train_loss=0.0005 | val_loss=9.1130 | val_acc=0.9582 | val_f1=0.2995 | val_auc=0.7981 | best_thr=0.05
Epoch 31/50 | train_loss=0.0001 | val_loss=9.2708 | val_acc=0.9580 | val_f1=0.2974 | val_auc=0.7869 | best_thr=0.05
Epoch 32/50 | train_loss=0.0003 | val_loss=8.8797 | val_acc=0.9589 | val_f1=0.3234 | val_auc=0.7874 | best_thr=0.05
Epoch 33/50 | train_loss=0.0001 | val_loss=9.0400 | val_acc=0.9587 | val_f1=0.3168 | val_auc=0.7835 | best_thr=0.05
Epoch 34/50 | train_loss=0.0003 | val_loss=8.8249 | val_acc=0.9590 | val_f1=0.3268 | val_auc=0.7925 | best_thr=0.05
Epoch 35/50 | train_loss=0.0000 | val_loss=10.2951 | val_acc=0.9564 | val_f1=0.2346 | val_auc=0.7840 | best_thr=0.05
Epoch 36/50 | train_loss=0.0002 | val_loss=9.4174 | val_acc=0.9579 | val_f1=0.2893 | val_auc=0.7849 | best_thr=0.05
Epoch 37/50 | train_loss=0.0001 | val_loss=9.8617 | val_acc=0.9572 | val_f1=0.2658 | val_auc=0.7865 | best_thr=0.05
Epoch 38/50 | train_loss=0.0005 | val_loss=9.5003 | val_acc=0.9579 | val_f1=0.2908 | val_auc=0.7864 | best_thr=0.05
Epoch 39/50 | train_loss=0.0003 | val_loss=9.5703 | val_acc=0.9579 | val_f1=0.2893 | val_auc=0.7854 | best_thr=0.05
Epoch 40/50 | train_loss=0.0001 | val_loss=9.5088 | val_acc=0.9580 | val_f1=0.2911 | val_auc=0.7870 | best_thr=0.05
```